



CRANFIELD UNIVERSITY



Deep Learning for Autonomous Decision Making (2025–26)

Deep Reinforcement Learning for Autonomous Driving

Name: Ebru Kilic

Student number: 483599

Lecturer: Dr. Adolfo Perrusquia

Word Count:2987

February 20, 2026

Contents

1	Introduction	2
2	Problem Formulation	2
2.1	State Space	2
2.2	Action Space	3
2.3	Reward Function	3
3	Methodology	3
3.1	Selection of the Learning Algorithm	3
3.2	State Representation Design	4
3.3	Action Space Design	4
3.4	Sensitivity Analysis Design	4
4	Proposed Inner Elements and Hyperparameters	4
4.1	Training Configuration	5
4.2	Evaluation Methodology	5
4.2.1	Policy Evaluation Protocol	5
4.2.2	Evaluation Metrics	5
5	Result and Discussion	6
5.1	Training Performance	6
5.2	Sensitivity Analysis	9
5.3	Generalisation Results	10
5.4	Discussion	12
6	Conclusion	12
	References	13
A	Code	14

1 Introduction

Autonomous driving is an important research area in modern transportation systems. The main goal is to develop vehicles that can move safely with little or no human intervention. This is difficult because driving environments are dynamic and unpredictable. A vehicle must constantly observe the road, make decisions, and react to changing conditions. Therefore, intelligent decision-making algorithms are required.

Reinforcement learning (RL) has become a popular method for solving control problems where an agent learns through interaction with an environment. In autonomous driving, deep reinforcement learning (DRL) is especially useful because it allows learning directly from visual inputs such as camera images. This removes the need for manually designed driving rules (El Sallab et al., 2017), (Mnih et al., 2015).

In this work, the CarRacing environment was used to simulate an autonomous driving task. The agent received visual observations of a racing track and learned how to steer, accelerate, and brake to complete laps efficiently while avoiding crashes. The aim was to model the problem as a Markov Decision Process (MDP) and design a reinforcement learning solution capable of stable navigation.

2 Problem Formulation

The driving task was formulated as a Markov Decision Process (MDP):

$$\mathcal{M} = (S, A, P, R, \gamma)$$

where S is the state space, A is the action space, P represents transition dynamics, R is the reward function, and γ is the discount factor.

The objective was to maximise the expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

2.1 State Space

The state space consists of processed visual observations. At each time step t , the state observation s_t is an element of the continuous state space:

$$s_t \in \mathcal{S} \subset \mathbb{R}^{6 \times 84 \times 84}$$

This representation includes spatial information from grayscale images and temporal information captured through stacking $n = 6$ consecutive frames. The original observations

were resized from 96×96 to 84×84 and converted to grayscale to reduce computational cost and improve training efficiency while preserving the key visual features of the track.

2.2 Action Space

A continuous action space $\mathcal{A}$ was defined to reflect realistic vehicle control. Each action vector $a_t \in \mathcal{A}$ lies within the bounded continuous space:

$$\mathcal{A} = [-1, 1] \times [0, 1] \times [0, 1]$$

where $a_t = [a_{steer}, a_{gas}, a_{brake}]^T$, representing steering, throttle, and braking inputs respectively.

Steering controlled the direction of the vehicle wheels and was defined in the range $[-1, 1]$, where negative values turned left and positive values turned right. Gas represented throttle control and was defined in the range $[0, 1]$, where higher values increased vehicle speed. Brake controlled deceleration and was also defined in the range $[0, 1]$, where larger values produced stronger braking force. These continuous controls allowed smooth and realistic vehicle motion (Farama Foundation, 2025).

2.3 Reward Function

The reward function was provided by the environment. Positive rewards were given when new track tiles were visited, while a small penalty of -0.1 was applied at each time step to encourage faster completion. The total reward was therefore determined by the balance between track progress and time (Farama Foundation, 2025).

3 Methodology

3.1 Selection of the Learning Algorithm

Proximal Policy Optimisation (PPO) was selected as the learning algorithm due to its robustness in continuous control tasks (Schulman et al., 2017). Its stability-oriented update mechanism makes it particularly suitable for autonomous driving, where unsafe or abrupt policy behaviours must be minimised (Inamdar et al., 2024). Furthermore, compared to off-policy algorithms like SAC, PPO often provides more stable training behaviour, while SAC is more sample efficient but harder to tune and often makes more unnecessary actions (Muzahid et al., 2021). Its core advantage lies in the use of a clipped surrogate objective function. This mechanism approximates Trust Region Policy Optimisation (TRPO) with significantly lower computational cost, effectively limiting excessive policy deviations and ensuring stable convergence (Schulman et al., 2017).

3.2 State Representation Design

Temporal frame stacking was applied to incorporate motion information into the state representation. Driving is a continuous process in which the vehicle state evolves over time, and a single visual observation cannot capture motion-related variables such as velocity or direction changes. By stacking multiple consecutive frames, the agent can observe temporal differences between observations and implicitly estimate speed and trajectory evolution, which is essential for stable vehicle control. This design is physically justified because key vehicle dynamics forces depend on velocity, which cannot be inferred from a single visual frame (Inamdar et al., 2024).

3.3 Action Space Design

A continuous action space was adopted to enable realistic vehicle control. Discrete actions would limit control precision and often lead to unstable driving behaviour (Lillicrap et al., 2015). This is because vehicle motion is constrained by physical limits such as tire friction and steering smoothness. Abrupt control changes may cause unstable trajectories or loss of grip, whereas gradual control adjustments allow smoother steering and acceleration, better reflecting realistic driving dynamics.

3.4 Sensitivity Analysis Design

A sensitivity analysis was performed using three configurations:

- Model A: $n_stack = 6$, $clip_range = 0.20$
- Model B: $n_stack = 4$, $clip_range = 0.20$
- Model C: $n_stack = 6$, $clip_range = 0.25$

4 Proposed Inner Elements and Hyperparameters

A convolutional neural network policy (CnnPolicy) was used to extract spatial features before producing continuous action outputs. PPO used an on-policy rollout buffer. In each update cycle, 1024 steps were collected across 4 parallel environments. Data were reused for multiple epochs and then discarded to maintain on-policy learning.

The clipping parameter was set to $\epsilon = 0.20$ to limit large policy changes. Large and sudden changes in control actions can destabilise vehicle motion in real driving systems. For example, abrupt steering or acceleration changes may lead to loss of traction or unstable trajectories. The clipping mechanism in PPO limits excessive policy updates,

ensuring smoother action changes over time. This results in more stable and physically realistic driving behaviour. The Adam optimiser was used with a linear learning rate decay starting from 3×10^{-4} . An entropy coefficient of 0.002 encouraged exploration (Stable Baselines3, 2026).

4.1 Training Configuration

Four parallel environments were used to increase data diversity and training speed. Random seeds were fixed to ensure reproducibility. Training time was limited to one hour to allow fair comparison between models.

4.2 Evaluation Methodology

4.2.1 Policy Evaluation Protocol

Policy evaluation was conducted using three different testing settings. First, stochastic performance was measured over 10 episodes using action sampling from the policy distribution to estimate potential performance. Second, deterministic stability was evaluated over 5 episodes using mean actions of policy to measure control consistency. Finally, generalisation ability was assessed using stochastic policy execution across 10 unseen tracks generated with different random seeds. A larger number of episodes was used for stochastic evaluation due to higher reward variance, while fewer episodes were sufficient for deterministic testing because of its stable and repeatable behaviour.

4.2.2 Evaluation Metrics

To rigorously assess the training performance and policy quality, multiple complementary metrics were selected.

- **Mean Reward and Learning Curve:** These metrics quantify task completion efficiency and convergence speed.
- **Kullback–Leibler (KL) divergence:** This metric measures the magnitude of policy updates and is commonly used in PPO to monitor policy stability. Large spikes indicate overly aggressive updates, which may lead to unstable or unsafe behaviour (Czworkowski et al., 2026).
- **Explained Variance:** It measures how well the value function predicts actual returns. Higher values indicate more accurate value estimation (Liu et al., 2025).
- **Value Loss:** It is the mean squared error between the predicted value and the observed return, measuring the accuracy of the critic network

- **Generalisation Performance:** Average reward and variance across unseen track seeds were used to evaluate policy robustness and adaptability to new environments, which is essential for real-world autonomous driving systems.

5 Result and Discussion

5.1 Training Performance

The training behaviour of the three PPO models was analysed using reward trends, learning curves, and optimisation metrics. These results were used to evaluate learning speed, convergence stability, and policy optimisation quality.

As shown in Figure 1, all models showed similar behaviour at the early stage of training. The average reward started from negative values and gradually increased as the agent learned basic driving skills such as staying on the track and following road direction. This phase represents initial exploration and policy formation (See Figure 2).

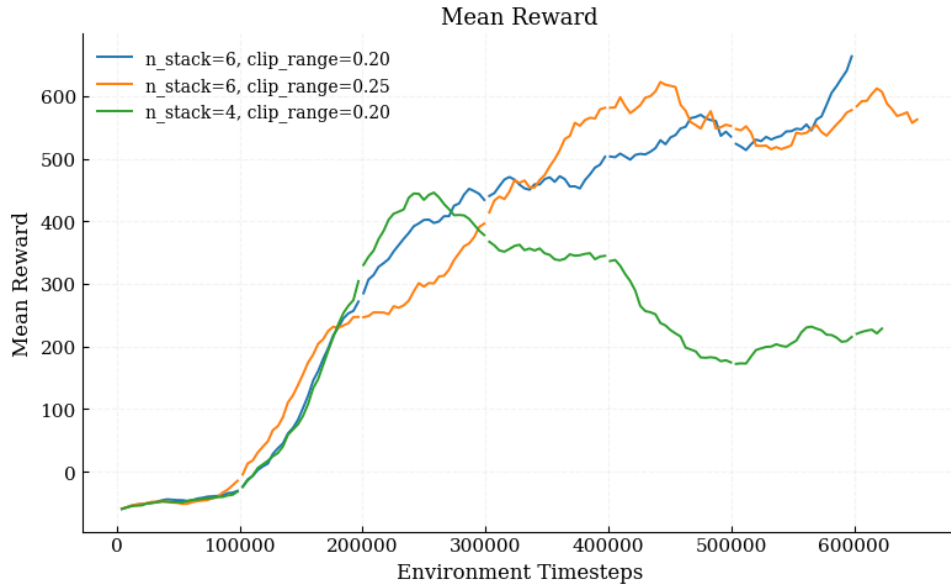


Figure 1 *Mean Reward Comparison Between Models*

Clear performance differences appeared after approximately 150,000 timesteps. The model with $n_stack = 6$ and $clip_range = 0.20$ achieved the highest final mean reward, exceeding 650 points. Its learning curve increased steadily, indicating stable policy improvement and consistent navigation behaviour .

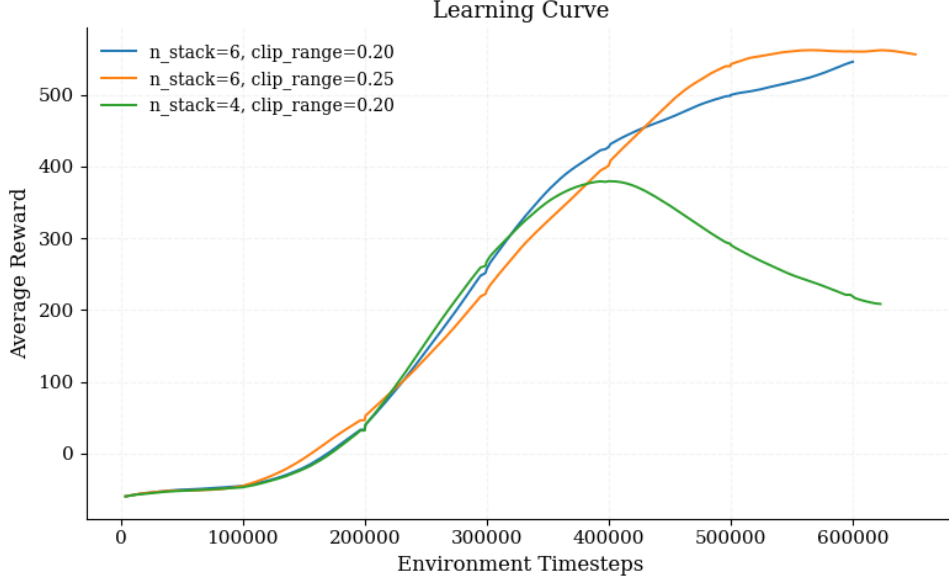


Figure 2 *Learning Curve Comparison Between Models*

The model with $n_stack = 6$ and $clip_range = 0.25$ also reached high rewards and temporarily outperformed the model with $n_stack = 6$ and $clip_range = 0.20$ between 350,000 and 500,000 timesteps. However, its learning curve showed larger fluctuations, suggesting that the larger clipping range allowed stronger policy updates that occasionally reduced stability. In contrast, the model with $n_stack = 4$ showed weaker performance. Although its reward increased during early training, a clear decline occurred after approximately 300,000 timesteps (see Figure 1). This indicates that limited temporal information reduced the agent’s ability to maintain stable control over long trajectories.

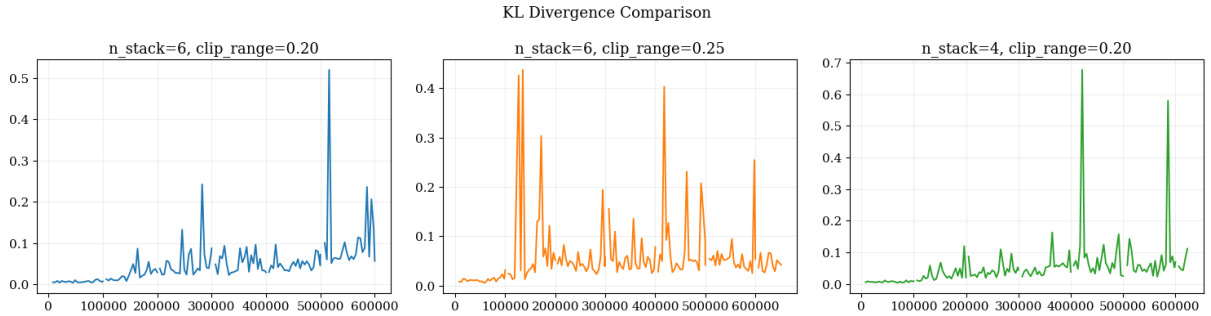


Figure 3 *KL Divergence Comparison Between Models*

The optimisation metrics supported these observations. As shown in Figure 3, KL divergence values remained generally low for all models, but occasional spikes were observed. More frequent spikes appeared in the model with a $clip_range = 0.25$, indicating unstable policy updates. However, the largest peak spike occurred in the model with $n_stack = 4$. This suggests that the limited temporal information led to noisier advan-

tage estimates, which in turn resulted in more aggressive policy updates. The model with $clip_range = 0.20$ and $n_stack = 6$ showed the most consistent KL behaviour overall. As seen in Figure 4, the value loss curves showed similar patterns. The model with $clip_range = 0.25$ had slightly more stable loss values, while the model with $clip_range = 0.20$ and $n_stack = 6$ showed moderate changes. The model with $n_stack = 4$ frames had the largest fluctuations, which indicates unstable value estimation due to limited motion information.

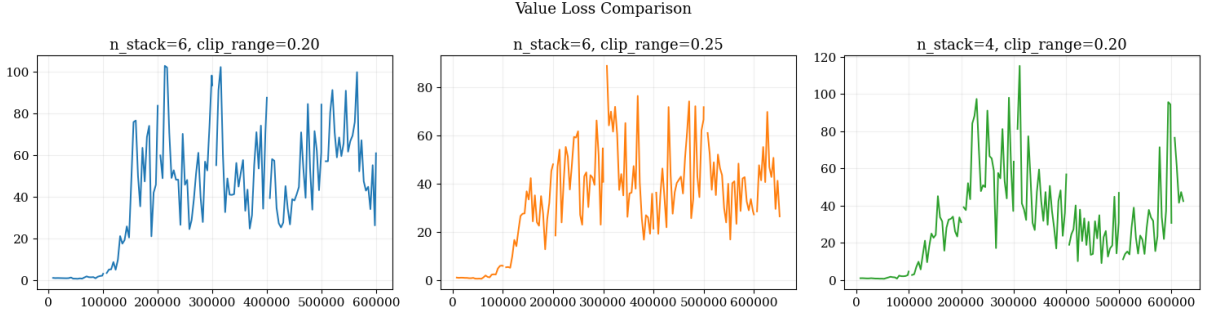


Figure 4 *Value Loss Comparison Between Models*

The explained variance metric rapidly approached values close to one for all models, which indicates successful learning of the value function. However, a significant drop was observed in the model with $n_stack = 4$, likely caused by reduced temporal information. (See Figure 5).

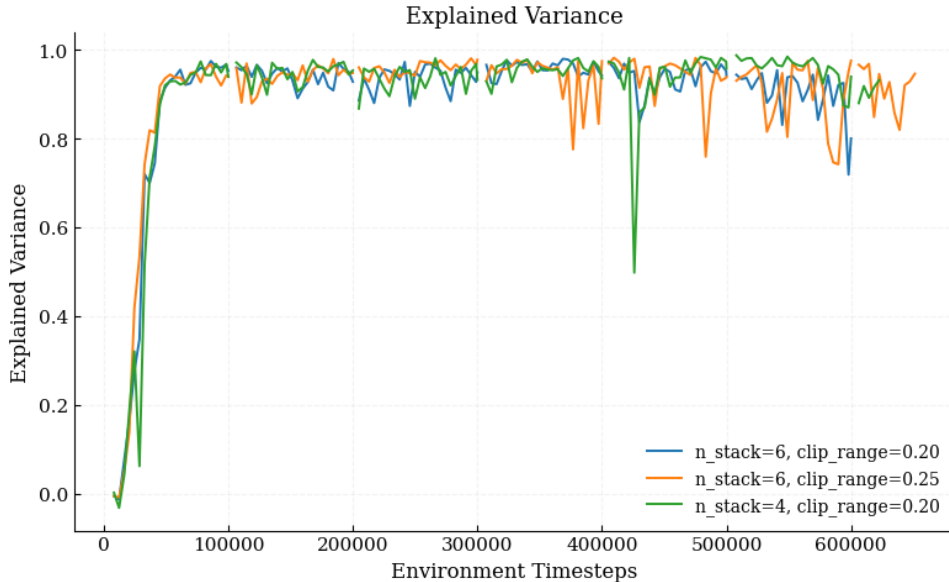


Figure 5 *Explained Variance Comparison Between Models*

Policy loss trends showed similar patterns. The model with $clip_range = 0.25$ maintained generally stable policy updates, despite an isolated spike early in training, while the other

configurations displayed occasional spikes, reflecting stronger policy corrections during training (See Figure 6).

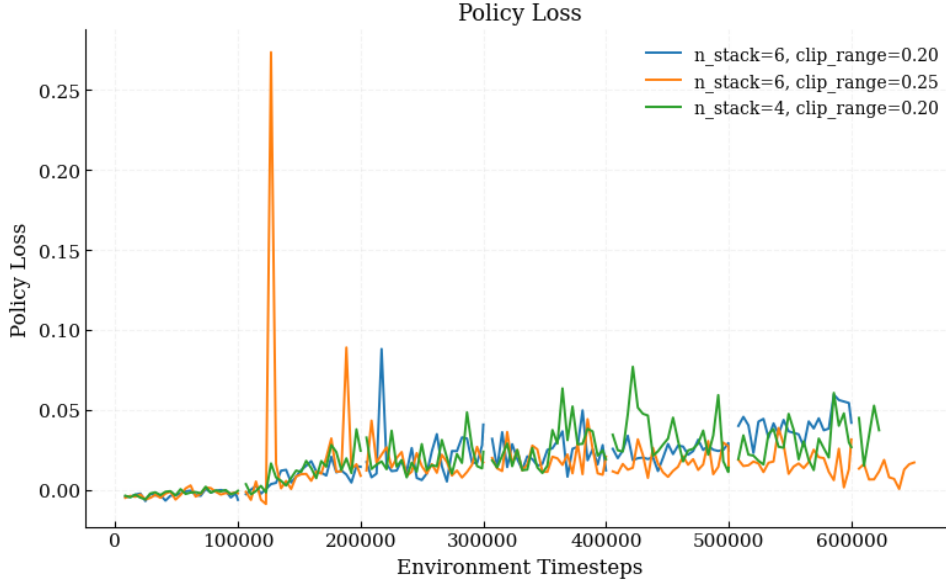


Figure 6 *Policy Loss Comparison Between Models*

5.2 Sensitivity Analysis

A sensitivity analysis was conducted to evaluate how small changes in temporal state representation and clipping range affected learning behaviour and final performance. Three PPO configurations were compared: different frame stacking sizes and clipping values.

First, the effect of frame stacking was analysed. When six frames were stacked, higher and more stable rewards were achieved. Both models with $n_stack = 6$ showed strong learning progress and maintained consistent performance throughout training. In contrast, the model with $n_stack = 4$ showed weaker behaviour. Although reward increased during early training, it began to decrease after approximately 400,000 timesteps. This indicates that insufficient temporal information reduced the agent’s ability to maintain stable control and predict motion dynamics.

Learning speed results further confirmed this observation. The highest reward gain rate was achieved by the configuration with $n_stack = 6$ and $clip_range = 0.20$, reaching 0.001209 reward increase per timestep. The model with $clip_range = 0.25$ showed slower learning (0.000955), while the configuration with $n_stack = 4$ had the lowest learning speed (0.000463).

Second, the influence of clipping range was evaluated. Increasing the $clip_range$ from 0.20 to 0.25 allowed larger policy updates, which produced faster early reward growth. However, this also introduced instability. The model with $clip_range = 0.25$ showed

larger fluctuations in KL divergence and policy loss. Several large spikes were observed during training, indicating aggressive policy changes. In comparison, the smaller clipping value maintained smoother optimisation behaviour. This instability was most evident in the deterministic testing phase. While the model with $clip_range = 0.20$ achieved a stable score of 602.5 model with the larger $clip_range = 0.25$ dropped significantly to 182, proving that aggressive updates harmed the policy’s reliability (See Figure 7).

The optimisation metrics strongly supported these findings. KL divergence remained relatively stable for the configuration with $n_stack = 6$ and $clip_range = 0.20$, while the other models showed frequent spikes. Similarly, value loss curves showed smoother convergence when sufficient temporal information was provided. Larger oscillations were observed when fewer frames were used.

The explained variance metric reached values close to one for all models, indicating successful value learning. However, temporary drops were observed in weaker configurations, especially when clipping was larger or frame stacking was smaller. These drops reflected short periods of unstable value estimation.

Overall, the sensitivity analysis demonstrated that performance was highly sensitive to both frame stacking and clipping range. Higher temporal information improved stability and reward consistency, while moderate clipping constraints ensured controlled policy updates and smoother learning dynamics.

5.3 Generalisation Results

The generalisation ability of the trained policies was evaluated using unseen racing tracks generated with 10 different random seeds. This test was important because a good autonomous driving policy should not only perform well on the training tracks but also adapt to new environments.

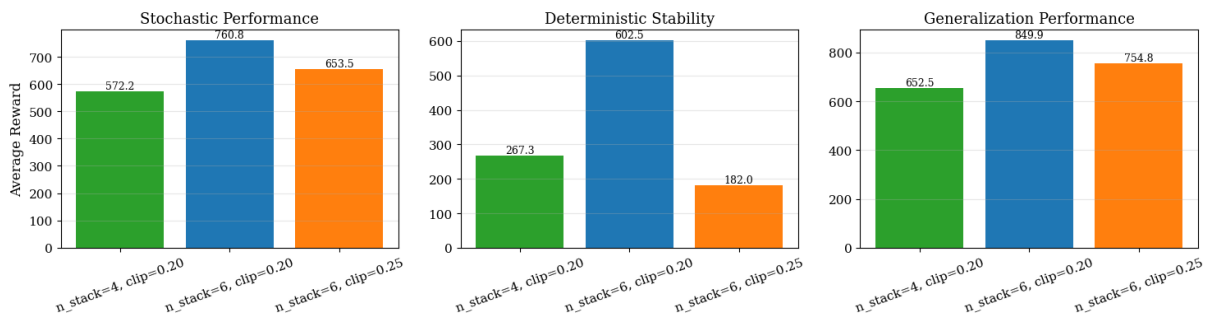


Figure 7 *Final Performance Comparison Between Models*

As seen in Figure 7, the results showed clear differences between the three model configurations. The model with $n_stack = 6$ and $clip_range = 0.20$ achieved the best

generalisation performance. It obtained an average reward of approximately 849.9 with a relatively small standard deviation of about 54.9. This indicates that the learned policy was both accurate and consistent across different unseen tracks. The generalisation capacity reached about 94% of the maximum solved score, which shows that the agent was able to maintain stable driving behaviour under new conditions. This value was calculated relative to the official environment reward threshold of 900 points defined in the Gymnasium CarRacing environment, which represents successful task completion.

Table 1 *Performance Comparison of PPO Configurations*

Model Configuration	Total Steps	Stochastic Reward	Deterministic Reward	Generalisation Mean	Generalisation Std
$n_stack = 6$, clip=0.20	600,000	760.8	602.5	849.9	54.9
$n_stack = 6$, clip=0.25	651,264	653.5	182.0	754.8	174.4
$n_stack = 4$, clip=0.20	622,592	572.2	267.3	652.5	82.5

The model with $n_stack = 6$ and $clip_range = 0.25$ also demonstrated good generalisation ability, with an average reward of about 754.8. However, a much larger performance variance was observed, with a standard deviation of approximately 174.4. This suggests that although high rewards were sometimes achieved, the policy was less stable and produced inconsistent driving behaviour across different tracks. This instability is consistent with earlier observations that larger clipping values allow stronger policy updates, which may reduce robustness.

The weakest generalisation performance was observed for the model with $n_stack = 4$ and $clip_range = 0.20$. This configuration achieved an average reward of approximately 652.5, with a moderate standard deviation of about 82.5. The reduced performance indicates that insufficient temporal information limited the agent’s ability to adapt to unseen track layouts. Without enough motion context, the policy struggled to maintain stable navigation when the environment changed.

These findings are consistent with the training performance results. Models with more stacked frames were able to capture temporal dynamics more effectively, which improved both learning stability and generalisation ability. In contrast, models with fewer stacked frames showed weaker performance because the agent had less information about vehicle motion.

Overall, the results demonstrate that temporal state representation plays a critical role in autonomous navigation. Sufficient frame stacking allowed the policy to generalise better to unseen environments, while appropriate clipping constraints ensured stable and consistent policy behaviour.

5.4 Discussion

The overall results show that both temporal state representation and clipping constraints had a strong impact on learning performance, stability, and generalisation ability.

It was observed that when six frames were used, more consistent rewards, smoother learning curves, and higher final performance were obtained. This improvement can be explained by the physical nature of driving. Motion perception requires information about speed, direction changes, and track curvature over time. With fewer frames, this temporal information was limited, which reduced control stability and caused performance decline during later training stages.

The clipping range was also found to play an important role in optimisation stability. When a larger clipping value was used, faster early learning was observed because larger policy updates were allowed. However, this also produced higher variability in performance. Large spikes in KL divergence and policy loss indicated that aggressive updates sometimes destabilised the policy. In contrast, a smaller clipping range provided more controlled optimisation and resulted in more reliable behaviour.

Some limitations of this study should be noted. Training time was restricted to one hour, which limited full convergence. In addition, the reward function was not modified. These factors may have influenced learning efficiency and performance.

6 Conclusion

In this work, an autonomous driving problem was modelled as a Markov Decision Process and solved using a PPO-based deep reinforcement learning approach. The results demonstrated that stable navigation can be learned directly from visual inputs in the CarRacing environment.

Among the tested configurations, the best overall performance was achieved when $n_stack = 6$ and $clip_range = 0.20$ were used. This configuration provided the most consistent learning behaviour, highest final reward, and strongest generalisation ability across unseen tracks.

These findings highlight the importance of balancing temporal state representation and policy update constraints when designing reinforcement learning systems for autonomous driving tasks.

References

- Czworkowski, C., Hornish, S., & Yasin, A. S. (2026). Proximal policy optimization with evolutionary mutations. *arXiv preprint arXiv:2601.14705*. <https://doi.org/10.48550/arXiv.2601.14705>
- El Sallab, A., Abdou, M., Perot, E., & Yogamani, S. (2017). Deep reinforcement learning framework for autonomous driving. *IS&T Electronic Imaging: Autonomous Vehicles and Machines*, 70–76. <https://doi.org/10.2352/ISSN.2470-1173.2017.19.AVM-023>
- Farama Foundation. (2025). *Car racing environment documentation*. Retrieved February 17, 2026, from https://gymnasium.farama.org/environments/box2d/car_racing/
- Inamdar, R., Sundarr, S. K., Khandelwal, D., Sahu, V. D., & Katal, N. (2024). A comprehensive review on safe reinforcement learning for autonomous vehicle control in dynamic environments. *Results in Engineering*, 22, 100810. <https://doi.org/10.1016/j.prime.2024.100810>
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2015). Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*. <https://doi.org/10.48550/arXiv.1509.02971>
- Liu, Q., Jiang, Z., Yang, H.-J., Khosravi, M., Waite, J. R., & Sarkar, S. (2025). Enhancing ppo with trajectory-aware hybrid policies. *arXiv preprint arXiv:2502.15968*. <https://doi.org/10.48550/arXiv.2502.15968>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>
- Muzahid, A. J. M., Kamarulzaman, S. F., & Rahman, M. A. (2021). Comparison of PPO and SAC algorithms towards decision making strategies for collision avoidance among multiple autonomous vehicles. *2021 International Conference on Software Engineering & Computer Systems and 4th International Conference on Computational Science and Information Management (ICSECS-ICOCSIM)*. <https://doi.org/10.1109/ICSECS52883.2021.00043>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*. <https://doi.org/10.48550/arXiv.1707.06347>
- Stable Baselines3. (2026). *Ppo — proximal policy optimization*. Retrieved February 17, 2026, from <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>

A Code

```
1 import gymnasium as gym
2 import numpy as np
3 import cv2
4 import time
5 import os
6 import random
7 import torch
8 from gymnasium import spaces
9 from stable_baselines3 import PPO
10 from stable_baselines3.common.vec_env import SubprocVecEnv,
    VecFrameStack, VecMonitor, DummyVecEnv
11 from stable_baselines3.common.callbacks import BaseCallback,
    EvalCallback
12 from stable_baselines3.common.logger import configure
13
14 # SEEDS
15 SEED = 42
16 EVAL_SEED = 999
17 np.random.seed(SEED)
18 random.seed(SEED)
19 torch.manual_seed(SEED)
20
21 # WRAPPERS
22 class SafeCarAction(gym.Wrapper):
23     def step(self, action):
24         if not isinstance(action, np.ndarray):
25             action = np.array(action, dtype=np.float32)
26         if action.ndim > 1:
27             action = action.reshape(-1)
28         return self.env.step(action)
29
30 class SkipZoomWrapper(gym.Wrapper):
31     def __init__(self, env, skip_steps=50):
32         super().__init__(env)
33         self.skip_steps = skip_steps
34
35     def reset(self, **kwargs):
```

```

36         obs, info = self.env.reset(**kwargs)
37         no_op = np.array([0.0, 0.0, 0.0], dtype=np.float32)
38         for _ in range(self.skip_steps):
39             obs, _, done, _, _ = self.env.step(no_op)
40             if done: break
41         return obs, info
42
43     class ImageProcessWrapper(gym.ObservationWrapper):
44         def __init__(self, env):
45             super().__init__(env)
46             self.observation_space = spaces.Box(low=0, high=255, shape
47                 =(1, 84, 84), dtype=np.uint8)
48
49         def observation(self, obs):
50             obs = obs.astype(np.uint8)
51             gray = cv2.cvtColor(obs, cv2.COLOR_RGB2GRAY)
52             resized = cv2.resize(gray, (84, 84), interpolation=cv2.
53                 INTER_LINEAR)
54             return resized[None, :, :]
55
56     # TRAINING TOOLS
57     def linear_schedule(initial_value):
58         def func(progress_remaining):
59             return progress_remaining * initial_value
60         return func
61
62     class TimeLimitCallback(BaseCallback):
63         def __init__(self, time_limit_secs, save_path, verbose=1):
64             super().__init__(verbose)
65             self.time_limit_secs = time_limit_secs
66             self.save_path = save_path
67             self.start_time = None
68
69         def _on_training_start(self) -> None:
70             self.start_time = time.time()
71
72         def _on_step(self) -> bool:
73             if (time.time() - self.start_time) > self.time_limit_secs:
74                 print(f"Time limit reached. Saving...")

```



```

73         self.model.save(os.path.join(self.save_path, "
           final_model_timeout"))
74         return False
75     return True
76
77 # MAIN LOGIC
78 def make_env(rank, seed=0):
79     def _init():
80         env = gym.make("CarRacing-v3", continuous=True, render_mode
           ="rgb_array")
81         env.reset(seed=seed + rank)
82         env = SafeCarAction(env)
83         env = SkipZoomWrapper(env)
84         env = ImageProcessWrapper(env)
85         return env
86     return _init
87
88 def train_and_evaluate():
89     save_path = "models_clip025"
90     log_path = "./logs_clip025/"
91     log_dir = "./logs_clip025/tensorboard/"
92     os.makedirs(save_path, exist_ok=True)
93     os.makedirs(log_path, exist_ok=True)
94     os.makedirs(log_dir, exist_ok=True)
95     new_logger = configure(log_dir, ["stdout", "csv", "tensorboard"
           ])
96
97     n_envs = 4
98     env = SubprocVecEnv([make_env(i, seed=SEED) for i in range(
           n_envs)], start_method='spawn')
99     env = VecMonitor(env)
100    env = VecFrameStack(env, n_stack=6, channels_order='first')
101
102    eval_env = DummyVecEnv([make_env(0, seed=EVAL_SEED)])
103    eval_env = VecMonitor(eval_env)
104    eval_env = VecFrameStack(eval_env, n_stack=6, channels_order='
           first')
105
106    eval_callback = EvalCallback(
107        eval_env,

```

```

108     best_model_save_path=save_path,
109     log_path=log_path,
110     eval_freq=25000,
111     n_eval_episodes=3,
112     deterministic=True,
113     verbose=1
114 )
115
116 time_callback = TimeLimitCallback(time_limit_secs=3600,
117     save_path=save_path)
118
119 model = PP0(
120     "CnnPolicy",
121     env,
122     verbose=1,
123     tensorboard_log=log_dir,
124     learning_rate=linear_schedule(0.0003),
125     n_steps=1024,
126     batch_size=256,
127     n_epochs=4,
128     ent_coef=0.002,
129     clip_range=0.20,
130     max_grad_norm=0.5,
131     device="auto",
132     seed=SEED,
133 )
134 model.set_logger(new_logger)
135 model.learn(total_timesteps=10_000_000, callback=[eval_callback
136     , time_callback])
137 model.save(os.path.join(save_path, "final_model_completed"))
138
139 # TEST PHASE
140 print("\nStarting Tests...")
141
142 try:
143     model = PP0.load(os.path.join(save_path, "best_model.zip"))
144     print("Best model loaded.")
145 except:
146     print("Using last checkpoint.")

```

```

146 for mode, rounds, name in [(False, 10, "Stochastic"), (True, 5,
147     "Deterministic")]:
148     print(f"\n---_{name}_Test_---")
149     rews = []
150     for i in range(rounds):
151         obs = eval_env.reset()
152         done, r_sum = False, 0
153         while not done:
154             action, _ = model.predict(obs, deterministic=mode)
155             obs, r, d, _ = eval_env.step(action)
156             r_sum += r[0]
157             done = d[0]
158             rews.append(r_sum)
159             print(f"Round_{i+1}:_{r_sum:.2f}")
160         print(f">>_{name}_Average:_{np.mean(rews):.2f}")
161
162 # GENERALIZATION TEST
163 print("\nGeneralization_Test_(10_Seeds)...")
164 unseen_seeds = [101, 202, 303, 404, 505, 606, 707, 808, 909,
165     1000]
166 unseen_rewards = []
167
168 for s in unseen_seeds:
169     temp_env = DummyVecEnv([make_env(0, seed=s)])
170     temp_env = VecFrameStack(temp_env, n_stack=6,
171         channels_order='first')
172     obs = temp_env.reset()
173     done, r_sum = False, 0
174     while not done:
175         action, _ = model.predict(obs, deterministic=False)
176         obs, r, d, _ = temp_env.step(action)
177         r_sum += r[0]
178         done = d[0]
179         unseen_rewards.append(r_sum)
180     temp_env.close()
181
182 print(f"\nFINAL_RESULTS")
183 print(f"Unseen_Tracks_Avg:_{np.mean(unseen_rewards):.2f}_+/_-{
184     np.std(unseen_rewards):.2f}")

```

```
181     print(f"Generalization Capacity: {(np.mean(unseen_rewards)
182           /900)*100:.1f}")
183
184     eval_env.close()
185     env.close()
186
187 if __name__ == "__main__":
188     train_and_evaluate()
```